- **Problem/Objective**

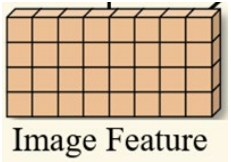
  “3D perception” – object detection, BEV segmentation

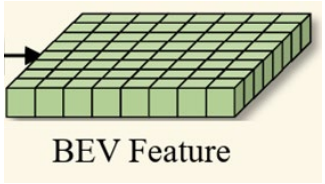
- **Contribution/Key Idea**

  1. **Feature Transportation Matrix(FTM)**
  2. **Prime Extraction**
  3. **Ring & Ray Decomposition**

View transformation

Multi camera features -> Bird's-Eye-View (BEV) features





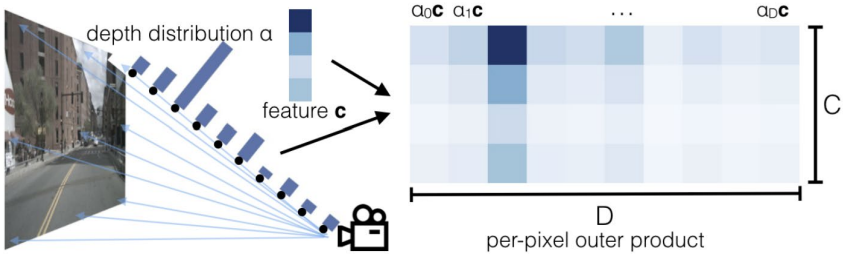
1. geometry-based method

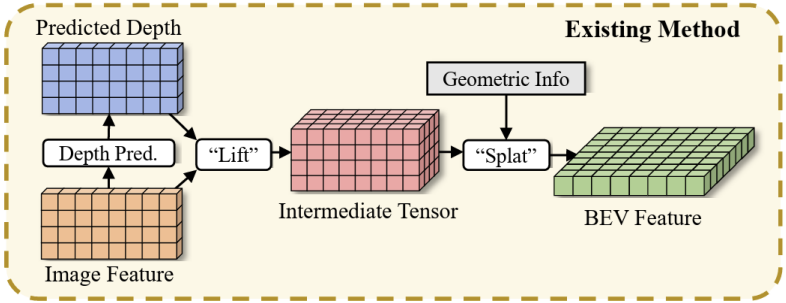
   - Lift-Splat

2. learning-based method

- **Background**

Lift-Splat





1. "lift" the F(image feature) into 3D space

$$F_{inter} = \{F_{inter}^{h,w}\}_{H_I,W_I} = \{\boldsymbol{F}^{\boldsymbol{h},\boldsymbol{w}} \cdot (\boldsymbol{D}^{\boldsymbol{h},\boldsymbol{w}})^T\}_{H_I,W_I} \quad (1)$$

1. The "splat" operation is then adopted, during which each feature vector is summed to a BEV grid according to geometric coordinates.

- **Background**

Lift-Splat

Problem
“splat” operation이 적합하지 않음 (cumsum trick - 모든 point에 대해 prefix sum을 계산한 다음 인덱스가 바뀌는 경계에서 이전 경계의 결과값을 빼는 방식)
The size of “lifted” multi-view image features is huge -> memory bottleneck of BEV models

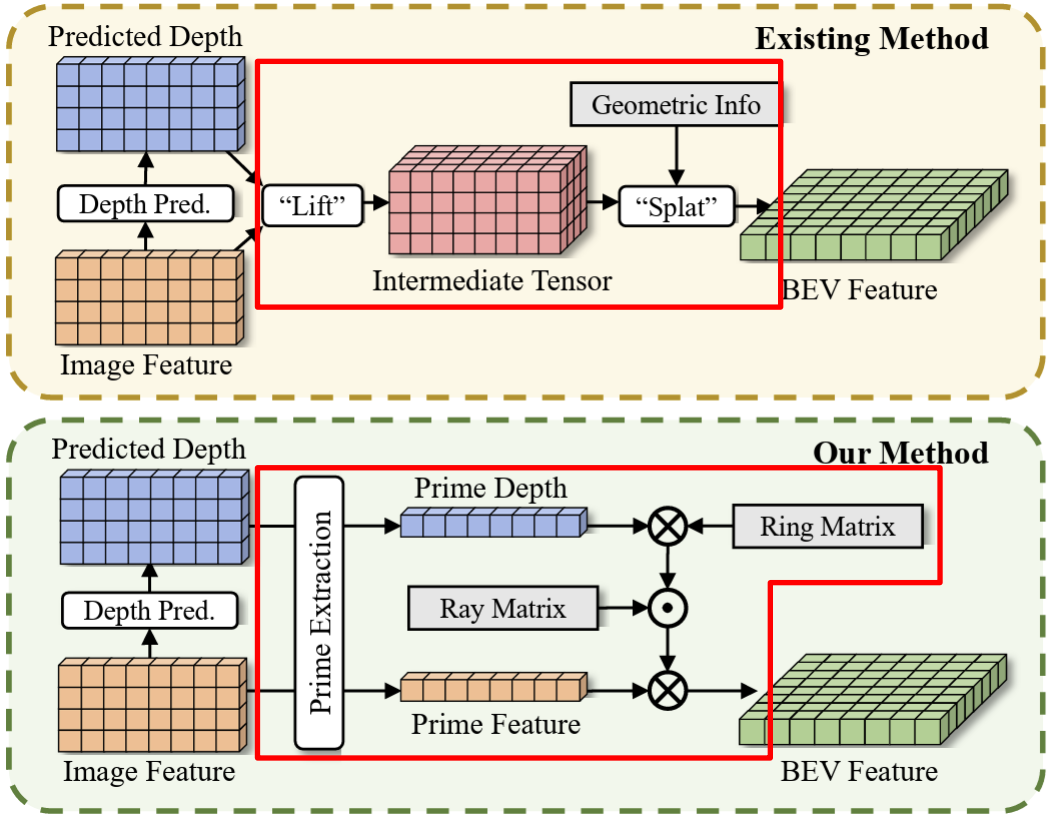
- **Background**

Lift-Splat

Problem	Solution
“splat” operation이 적합하지 않음 (cumsum trick)	- Feature Transporting Matrix (FTM) - Prime Extraction - Ring & Ray Decomposition
The size of “lifted” multi-view image features is huge -> memory bottleneck of BEV models	

MatrixVT: Efficient Multi-Camera to BEV Transformation for 3D Perception

ICCV 2023



- Feature Transporting Matrix (FTM)
- Prime Extraction
- Ring & Ray Decomposition

김범준

- **Architecture Diagram**

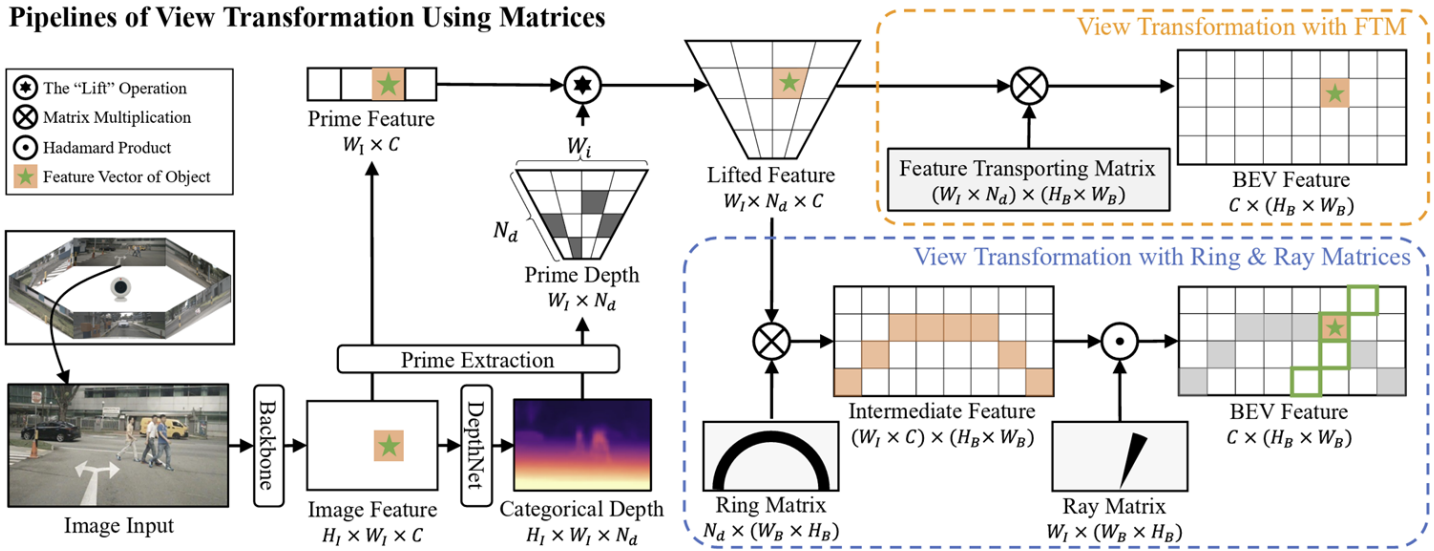
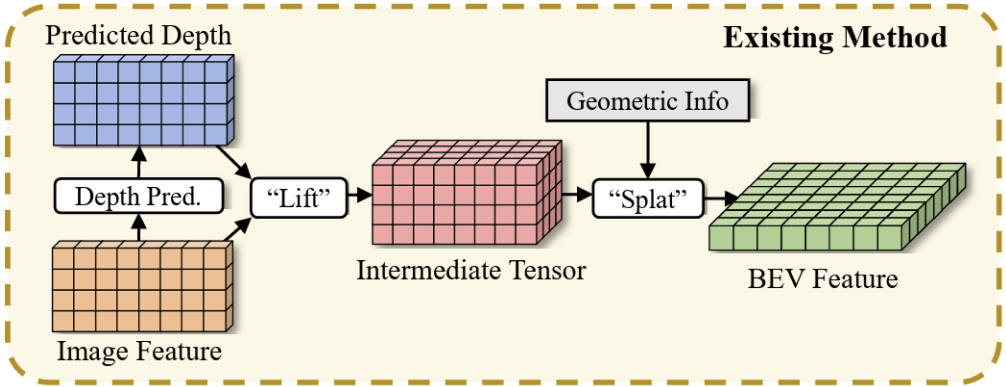


Figure 4. View Transformation using matrices. We first extract features from the image and predict categorical depth for each pixel. The obtained feature and depth are sent to the Prime Extraction module for compression. With the compressed Prime Feature and the Prime Depth, we "lift" the 1D Prime Feature into 2D space using Prime Depth. The "lifted feature" is then transformed into the BEV feature by MatMul with FTM or the decomposed Ring & Ray matrices.

- **Feature Transporting Matrix (FTM)**

Motivation

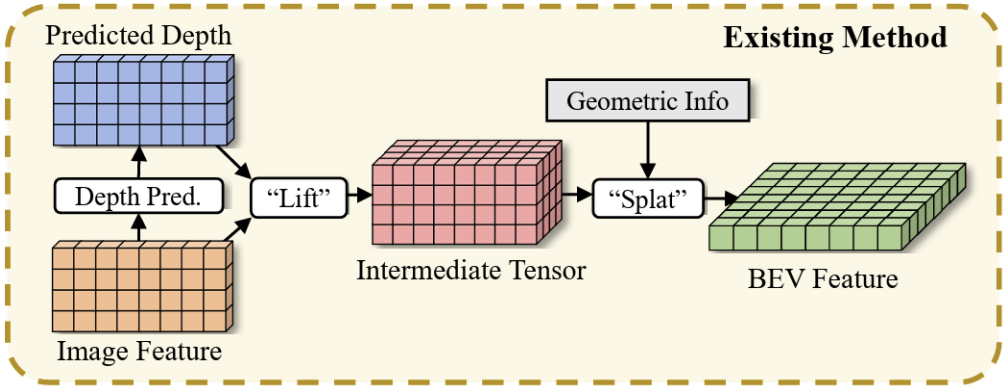
"splat" operation은 fixed mapping between the intermediate tensor and BEV grids



- **Feature Transporting Matrix (FTM)**

Motivation

"splat" operation은 fixed mapping between the intermediate tensor and BEV grids
= "lifted" feature



$$\mathrm{F_{BEV}} = \mathrm{M_{FT}} \cdot \mathrm{F_{inter}}$$

$$\mathrm{M_{FT}} \in \mathbb{R}^{(H_B \times W_B) \times (N_c \times H_I \times W_I \times N_d)}$$

$$\mathrm{F_{inter}} \in \mathbb{R}^{(N_c \times H_I \times W_I \times N_d) \times C}$$

$$\mathrm{F_{BEV}} \in \mathbb{R}^{(H_B \times W_B) \times C}$$

김범준

- **Feature Transporting Matrix (FTM)**

$$\mathrm{F}_{\mathrm{BEV}} = \boxed{\mathrm{M}_{\mathrm{FT}}} \cdot \mathrm{F}_{\mathrm{inter}}$$

- **Feature Transporting Matrix (FTM)**

$$\mathrm{F}_{\mathrm{BEV}} = \mathrm{M}_{\mathrm{FT}} \cdot \mathrm{F}_{\mathrm{inter}}$$

- Sparse FTM

-> Prime Extraction & Ring and Ray Decomposition

- **Prime Extraction**

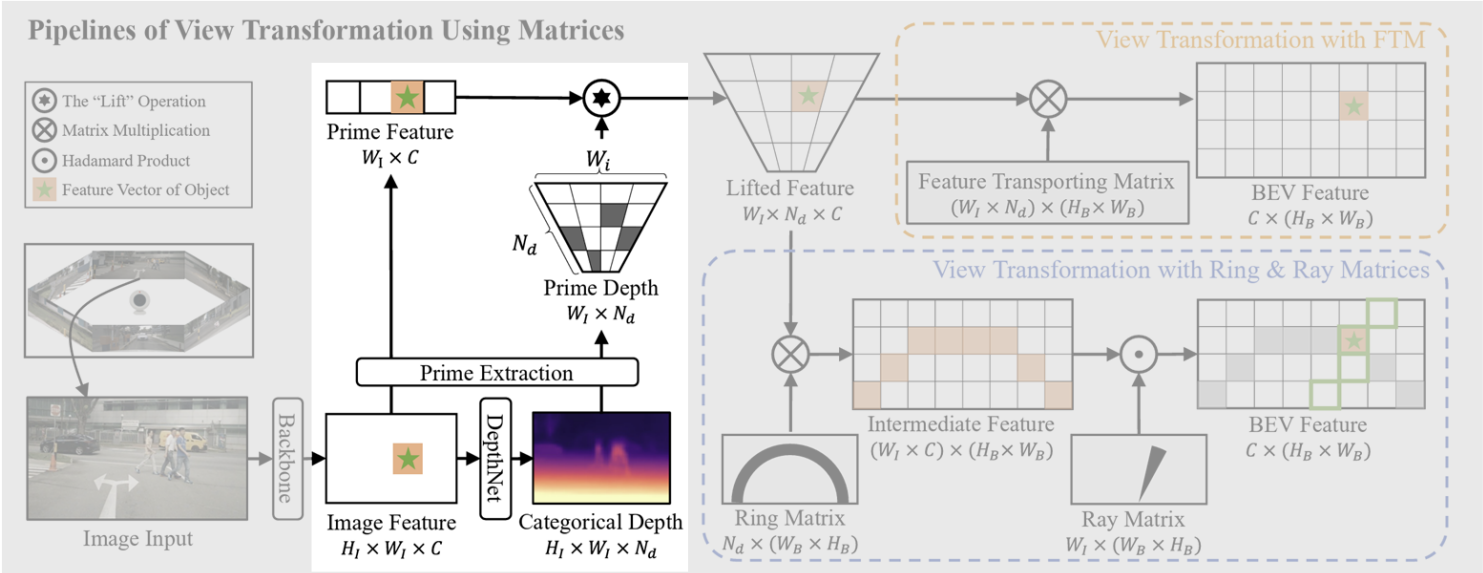


Figure 4. View Transformation using matrices. We first extract features from the image and predict categorical depth for each pixel. The obtained feature and depth are sent to the Prime Extraction module for compression. With the compressed Prime Feature and the Prime Depth, we "lift" the 1D Prime Feature into 2D space using Prime Depth. The "lifted feature" is then transformed into the BEV feature by MatMul with FTM or the decomposed Ring & Ray matrices.

- **Prime Extraction**

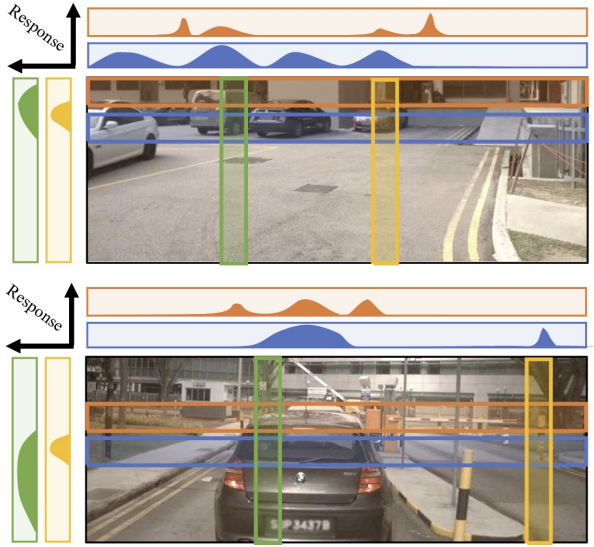


Figure 2. Response strengths along vertical and horizontal dimensions. For autonomous driving, the variance of response on the width dimension is higher than height dimension.

Sparsity를 줄이는 방법 : F_{inter} size를 줄이기

-> Feature compression

Motivation

- Image feature에서 dimension이 가지고 있는 정보량 비교

  height dimension < width dimension

-> Image features를 height dimension에 대해 압축하자.

Prime Extraction

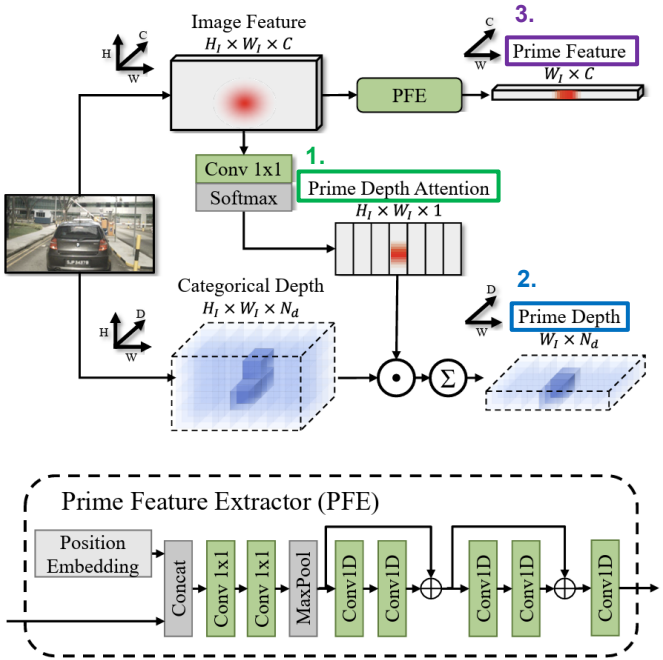


Figure 3. The Prime Extraction module. The categorical depth is reduced guided by Prime Depth Attention, while the image feature is reduced by Prime Feature Extractor (PFE), which consists of MaxPooling and convolutions for refinement.

1. Image Feature에서 height별로 가장 중요한 feature 정보가 있는 곳을 찾음 – **Prime Depth Attention**

2. 예측한 depth distribution(= categorical depth distribution)값과 1.을 weighted sum하여 Prime Depth를 구함.
– **Prime Depth**

3. Image Feature에서 convolution과 높이 축에 대한 Maxpool을 수행하여 높이 축에 대한 Prime Feature를 구함. – **Prime Feature**

전) $\mathrm{M_{FT}} \in \mathbb{R}^{(H_B \times W_B) \times (N_c \times H_I \times W_I \times N_d)}$

후) $\mathrm{M'_{FT}} \in \mathbb{R}^{(H_B \times W_B) \times (N_c \times W_I \times N_d)}$

김범준

MatrixVT: Efficient Multi-Camera to BEV Transformation for 3D Perception

ICCV 2023

- **"Ring and Ray" Decomposition**

Sparsity를 줄이는 방법 -> Matrix decomposition

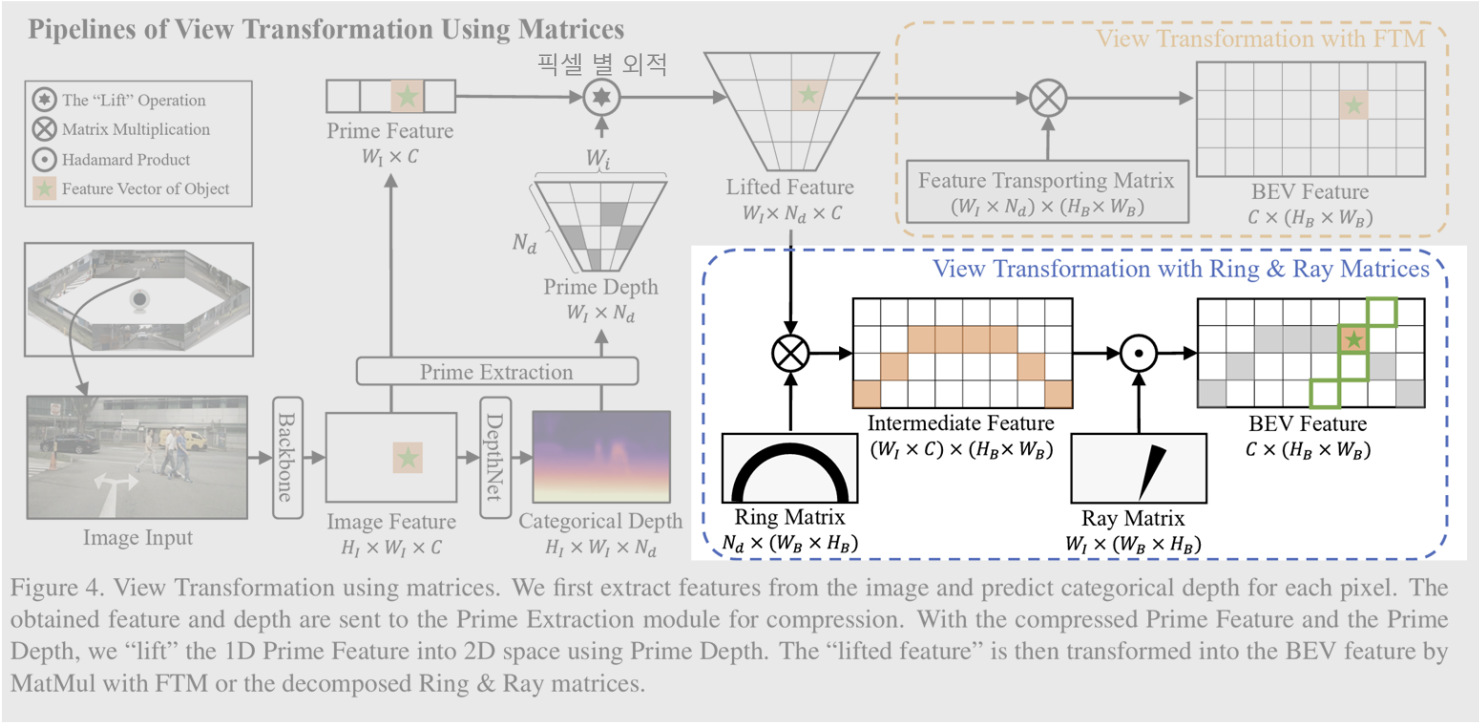


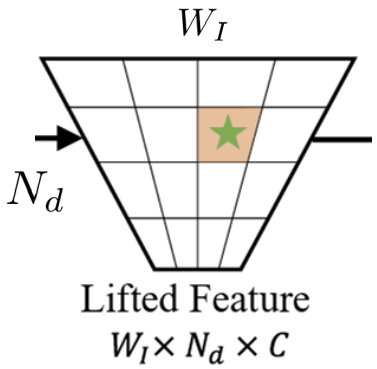
Figure 4. View Transformation using matrices. We first extract features from the image and predict categorical depth for each pixel. The obtained feature and depth are sent to the Prime Extraction module for compression. With the compressed Prime Feature and the Prime Depth, we "lift" the 1D Prime Feature into 2D space using Prime Depth. The "lifted feature" is then transformed into the BEV feature by MatMul with FTM or the decomposed Ring & Ray matrices.

김범준

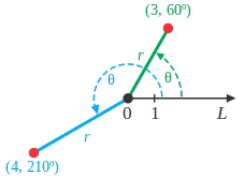
MatrixVT: Efficient Multi-Camera to BEV Transformation for 3D Perception

ICCV 2023

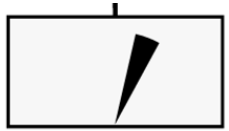
- **"Ring and Ray" Decomposition**



[Polar coordinate]



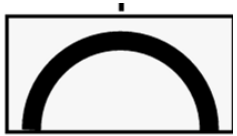
W_I -> **direction**



Ray Matrix
$W_I \times (W_B \times H_B)$

$\mathrm{M}_{\mathrm{Ray}} \in \mathbb{R}^{W_I \times (H_B \times W_B)}$

N_d -> **distance**



Ring Matrix
$N_d \times (W_B \times H_B)$

$\mathrm{M}_{\mathrm{Ring}} \in \mathbb{R}^{N_d \times (H_B \times W_B)}$

M'_{FT}

$W_I \times N_d \times H_B \times W_B$ → M'_{FT}

$(W_I + N_d) \times H_B \times W_B$

김범준

- **"Ring and Ray" Decomposition**

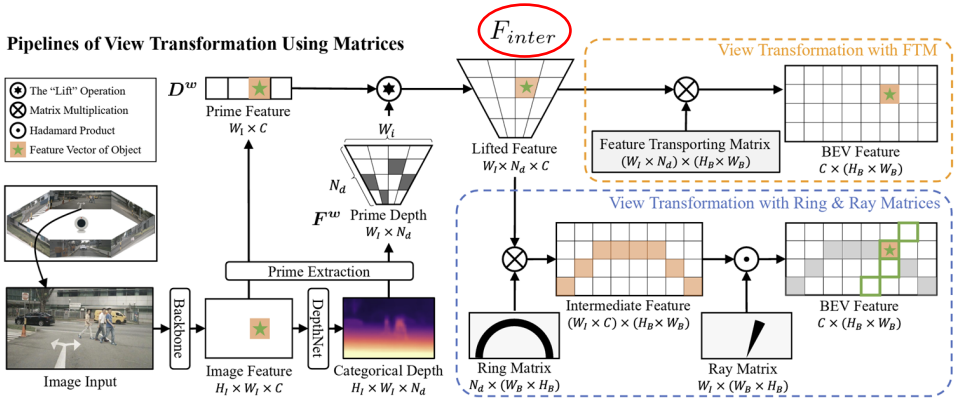


Figure 4. View Transformation using matrices. We first extract features from the image and predict categorical depth for each pixel. The obtained feature and depth are sent to the Prime Extraction module for compression. With the compressed Prime Feature and the Prime Depth, we "lift" the 1D Prime Feature into 2D space using Prime Depth. The "lifted feature" is then transformed into the BEV feature by MatMul with FTM or the decomposed Ring & Ray matrices.

$$F_{inter} = \{F^w_{inter}\}_{W_I} = \{\boldsymbol{F^w} \otimes (\boldsymbol{D^w})^{\boldsymbol{T}}\}_{W_I} \quad (3)$$

$$\mathrm{F}^*_{\mathrm{inter}} = \mathrm{M}_{\mathrm{Ray}} \odot (\mathrm{M}_{\mathrm{Ring}} \cdot \mathrm{F}_{\mathrm{inter}}) \quad (4)$$

$$F_{BEV} = \sum_w F^{*w}_{inter} \quad (5)$$

$$F^*_{inter} \in \mathbb{R}^{W_I \times C \times H_B \times W_B}$$

$$\mathrm{F}_{\mathrm{BEV}} = \mathrm{F} \cdot (\mathrm{M}_{\mathrm{Ray}} \odot (\mathrm{M}_{\mathrm{Ring}} \cdot \mathrm{D})) \quad (6)$$

김범준

- **"Ring and Ray" Decomposition**

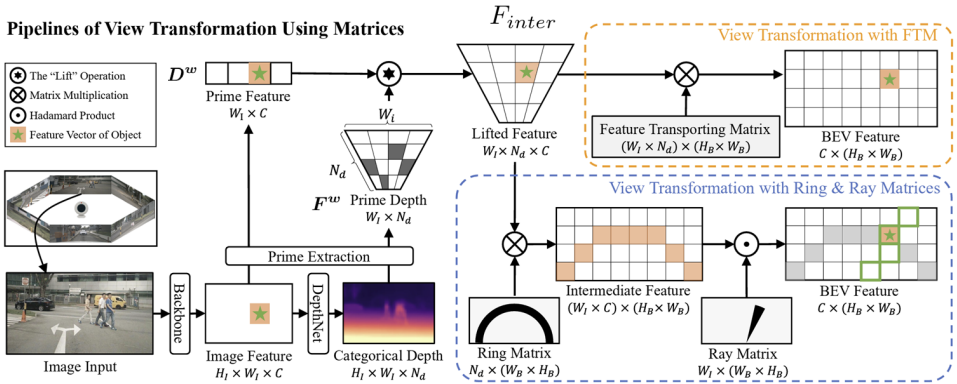


Figure 4. View Transformation using matrices. We first extract features from the image and predict categorical depth for each pixel. The obtained feature and depth are sent to the Prime Extraction module for compression. With the compressed Prime Feature and the Prime Depth, we "lift" the 1D Prime Feature into 2D space using Prime Depth. The "lifted feature" is then transformed into the BEV feature by MatMul with FTM or the decomposed Ring & Ray matrices.

$$\mathrm{F_{BEV}} = \mathrm{F} \cdot (\mathrm{M_{Ray}} \odot (\mathrm{M_{Ring}} \cdot \mathrm{D})) \qquad (6)$$

	FLOPs	Memory footprint
전)	$2 \times W_I \times C \times N_d \times H_B \times W_B$	$W_I \times N_d \times H_B \times W_B$
후)	$2(C + N_d + 1) \times W_I \times H_B \times W_B$	$(W_I + N_d) \times H_B \times W_B$

김범준

- **Experiments**

Method	Backbone	Resolution	MF	mAP↑	mATE↓	mASE↓	mAOE↓	mAVE↓	mAAE↓	NDS↑
BEVDet [6]	Res-50	256 × 704	×	0.286	0.724	0.278	0.590	0.873	0.247	0.372
PETR [11]	Res-50	384 × 1056	×	0.313	0.768	0.278	0.564	0.923	0.225	0.381
BEVDepth [9]	Res-50	256 × 704	×	**0.337**	0.646	0.271	0.574	0.838	0.220	0.414
MatrixVT	Res-50	256 × 704	×	0.336	0.653	0.271	0.473	0.903	0.231	**0.415**
BEVDet [6]	Res-101	384 × 1056	×	0.330	0.702	0.272	0.534	0.932	0.251	0.396
FCOS3D [28]	Res-101	900 × 1600	×	0.343	0.725	0.263	0.422	1.292	0.153	0.415
PETR [11]	Res-101	512 × 1408	×	0.357	0.710	0.270	0.490	0.885	0.224	0.421
BEVFormer [10]	R101-DCN	900 × 1600	×	0.375	0.725	0.272	0.391	0.802	0.200	0.448
MatrixVT	Res-101	512 × 1408	×	**0.396**	0.577	0.261	0.397	0.870	0.207	**0.467**
BEVFormer [10]	R101-DCN	900 × 1600	✓	0.416	0.673	0.274	0.372	0.394	0.198	0.517
BEVDet4D [5]	Swin-B	640 × 1600	✓	0.421	0.579	0.258	0.329	0.301	0.191	0.545
BEVDepth [9]	V2-99	512 × 1408	✓	0.464	0.528	0.254	0.350	0.354	0.198	**0.564**
MatrixVT	V2-99	512 × 1408	✓	**0.466**	0.535	0.260	0.380	0.342	0.198	0.562

Table 1. Experimental results of object detection on the nuScenes val set. The "MF" indicates multi-frame fusion.

- **Experiments**

Method	Backbone	mAP↑	mAPH↑	mAPL↑
BEVDepth [9]	Res-50	0.290	0.208	0.253
MatrixVT	Res-50	0.304	0.215	0.259

Table 2. Performance of object detection on Waymo val set.

- **Experiments**

Method	IoU-Drive↑	IoU-Lane↑	IoU-Vehicle↑
LSS [17]	0.729	0.200	0.321
FIERY [4]	-	-	0.382
M^2BEV [30]	0.759	0.380	-
BEVFormer [10]	0.775	0.239	0.467
BEVDepth-Seg	0.827	**0.464**	0.450
MatrixVT	**0.835**	0.448	**0.462**

Table 3. Experimental results of BEV segmentation on the nuScenes val set. We implement map segmentation on the BEVDepth by placing a segmentation head on the BEV feature.

- **Experiments**

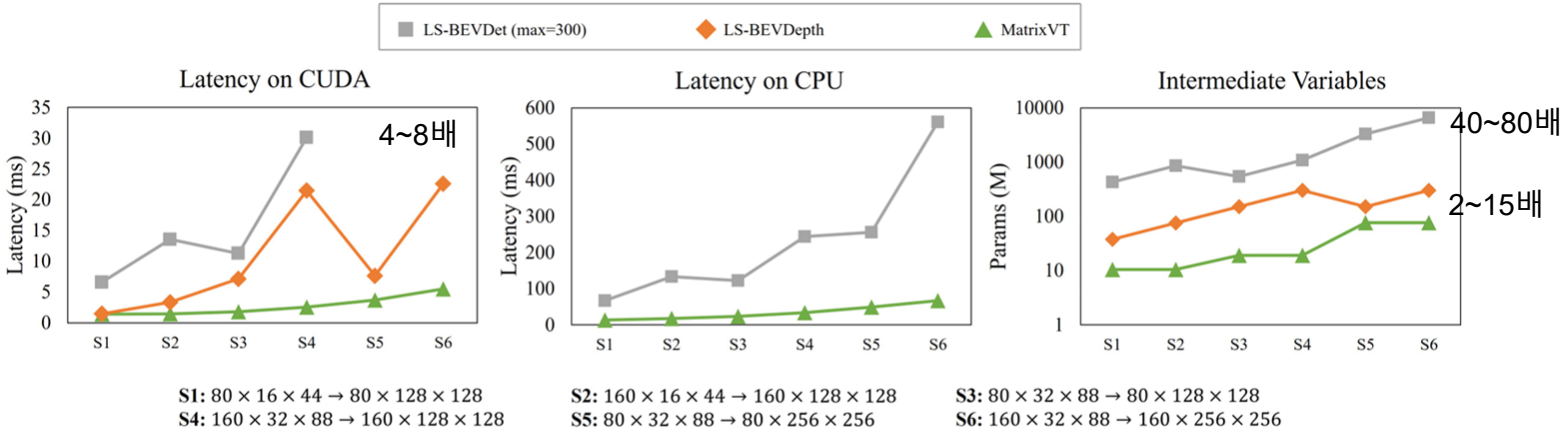


Figure 5. Latency (left, middle) and the number of intermediate parameters (right) of MatrixVT and two implementations of Lift-Splat, the S1∼S6 denote transformation settings represented by image feature size ($C \times H_I \times W_I$) to BEV feature size ($C \times H_B \times W_B$). Note that the LS-BEVDet raises Out Of Memory error under S5 and S6 on CUDA. The LS-BEVDepth is not available without the CUDA platform.

- **Experiments**

Backbone	PE	RR	NDS	Mem.	Lat. (ms)
Res-50			0.416	528M	1.79
Res-50	✓		0.416	94M	1.42
Res-50	✓	✓	0.415	**83M**	**1.33**
V2-99			0.564	2.14G	8.85
V2-99	✓		0.563	**405M**	**2.04**
V2-99	✓	✓	0.562	498M	3.19

Table 4. Effects of using Prime Extraction (PE) and Ring & Ray FTM (RR) on BEVDepth. The memory consumption and latency reported is the memory footprint and time duration on PyTorch *during View Transformation (VT)*.

- **Experiments**

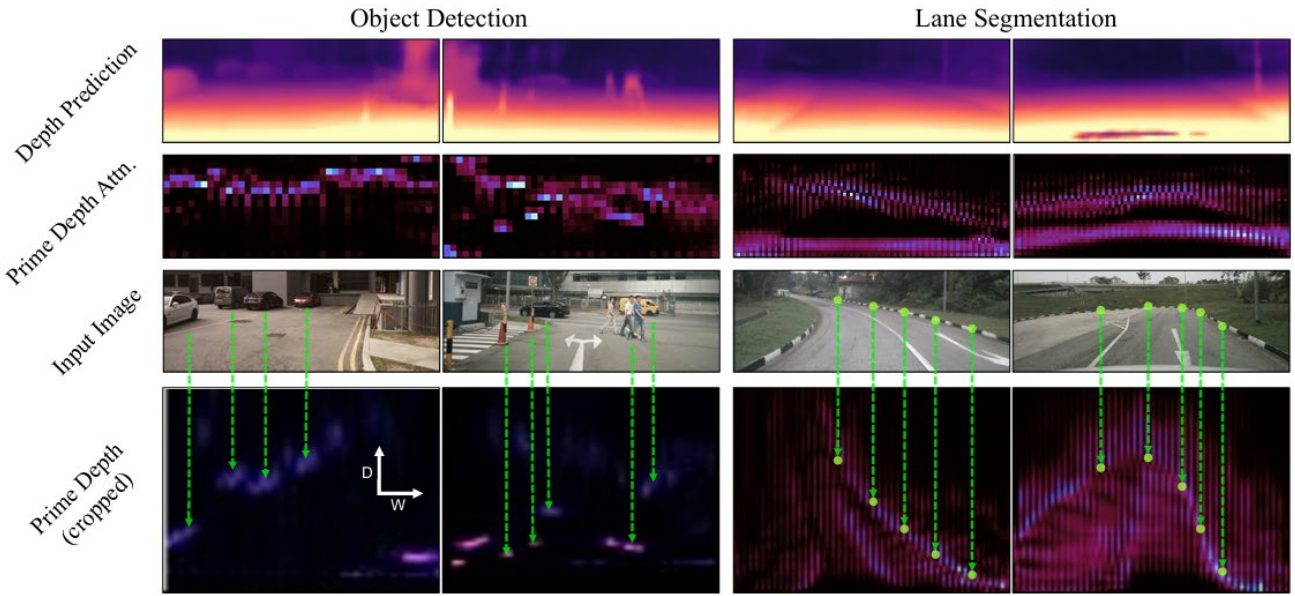


Figure 6. Depth Prediction, Prime Depth Attention, and Prime Depth for object detection and lane segmentation task. The categorical depth prediction is illustrated using distances with largest probability, while the Prime Depth is illustrated by raw probability values.